Recovery Strategy Evaluation for Tool-Using LLM Agents

Course Midterm Report · Oludolapo Adegbesan · October 12, 2026

1. Project Overview

Project title: Recovery Strategy Evaluation for Tool-Using LLM Agents on tau-bench

Problem and purpose: Large language model (LLM) agents that call external tools are vulnerable to transient tool failures — a tool may return empty data, corrupted identifiers, or an error response. When this happens, the agent may proceed on bad information and fail the task entirely. Despite the growing use of tool-calling agents in real applications, there is little empirical work systematically comparing how different recovery strategies perform across different fault types. This project addresses that gap.

Goals and objectives:

1. Design and implement a controlled fault injection framework layered over the official tau-bench retail benchmark, allowing precise injection of three failure modes without modifying the benchmark's tasks, evaluator, or agent.
2. Implement and evaluate six recovery strategies — no recovery, restart, visible retry, checkpoint, source rollback, and selective — measuring task success rate, source localization accuracy, and computational overhead per strategy.
3. Derive and test three additional strategies from recent literature (rollback with reflection, verbal correction, adaptive) to extend the comparison.
4. Identify which recovery strategies are most effective per fault type and understand why, using trajectory analysis.

Expected final outcome: A written research paper with quantitative comparison tables, trajectory-level analysis, and concrete guidance on when each recovery strategy should be applied. The primary measure is official tau-bench reward conditioned on the fault having fired; secondary measures include exact source localization accuracy, attempt steps, model calls, and wall time.

2. Work Completed: Weeks 1–7

Weeks 1–2 (late August – early September): Literature Review and Research Design

Conducted a survey of existing work on LLM agent robustness and tool-failure recovery, covering tau-bench (Yao et al. 2024), AgentBench, and recent arXiv preprints on agent self-correction and rollback (e.g., "Rollback the World, Keep the Reflection" arXiv 2609.18304; "Retry, Switch, or Abstain?" arXiv 2608.11977; "Self-Healing Framework" arXiv 2605.06737; "Outcome Monitors" arXiv 2608.19303). Formulated the core research question: *which recovery strategy best restores task success after a controlled tool failure, and does the answer depend on fault type?*

Set up the tau-bench environment (Python 3.12, tau2 library pinned at a specific verified commit) and validated end-to-end execution with the retail task suite and Gemini LLM backend.

Weeks 3–4 (September – early October): Adapter Implementation

Designed and implemented `Backend/research/tau_recovery.py` — the RecoveryController, a custom adapter wrapping tau-bench's official retail simulation. Key design decisions:

- **Fault injection:** Three fault types injected at the first successful `get_user_details` call: `empty_read` (returns `{}`), `wrong_user_id` (corrupts the user ID field to `__controlled_invalid_user__`), `tool_error` (returns a transient timeout error with `error=True`).
- **Detection:** Deterministic source detection by re-executing tool calls against the current database state and comparing canonical outputs. The earliest mismatch after at least one downstream step is identified as the fault source.
- **DAG dependency tracking:** Builds an explicit directed acyclic graph of tool calls and model steps. Each recovery strategy uses this graph to determine which nodes to invalidate.
- **Six recovery strategies implemented:** no_recovery (no action), restart (full conversation restart), visible_retry (one-step rollback), checkpoint (rollback to nearest 4-step boundary), source_rollback (rollback to fault source), selective (rollback to source, reuse verified downstream results).

- **Verified cache:** After selective recovery, downstream tool results that pass re-validation against the restored database state are cached and reused, avoiding redundant re-execution.
- **Protocol file:** Designed and froze `docs/research/tau-recovery-protocol.json` specifying all experimental parameters, measurement definitions, and analysis rules before any live trials.
- **Matrix runner:** Implemented `Backend/research/tau_recovery_matrix.py` — a hash-guarded matrix executor that records all results as structured JSON cell files and aborts if the adapter or protocol changes mid-run.

Weeks 5–6 (early–mid October): Experimental Execution (Phases 1–3)

Phase 1 — Seeds 301–302, empty_read fault, all 6 strategies (task 0): Executed via the matrix runner. Discovered that source rollback consistently achieves reward=1.0 across all three seeds (300, 301, 302) while selective fails on seed 301 (reward=0.0) despite identical task input. Documented the checkpoint brittleness pattern: checkpoint selects a fixed 4-step boundary independent of where the fault occurs, which can land in an alignment-sensitive position and cause failure. Each phase's adapter version is logged in the research log for reproducibility.

Phase 2 — Seed 300, all faults, all strategies (task 0): Extended execution to wrong_user_id and tool_error faults. First evidence that wrong_user_id is recoverable by any detection-based strategy. First tool_error results: selective=1.0 (seed 300), all others with fault exercised=0.0. Discovered that many tool_error cells had `fault_exercised=False` — the agent bypassed `get_user_details` via `find_user_id_by_name_zip`, so the fault never fired.

Phase 3 — Seeds 301–302, wrong_user_id + tool_error (task 0): Completed in two runs (aborted and resumed after accidental protocol JSON edit mid-run triggered the hash guard — see Challenges). Final cell count after Phase 3: 87 cells across 9 matrix runs.

Week 7 (October 12): Analysis, Dashboard, and Literature-Driven Strategy Design

- **Trajectory analysis:** Read all tool_error exercised cells to explain why selective=1.0 on seed 300 but selective=0.0 on seed 301. Finding: the difference is detection timing, not mechanism. On seed 300, selective detected the fault at step 14 (after product lookups were complete); other strategies detected at steps 8–10 (earlier in the task). After rollback, selective had very few steps to re-execute and the agent completed the exchange before the simulator ended the conversation. Strategies that detected

earlier had more post-recovery work; the conversation hit the ###STOP### termination before exchange_items was called, yielding reward=0.0 even though the recovery mechanism itself was correct.

- **Results dashboard:** Built `Backend/research/build_results_table.py` to scan all matrix directories and generate a self-contained interactive HTML dashboard with reward grid and fault coverage tables. Published at Results Dashboard.
- **Overhead analysis:** Extracted model_calls and wall time per strategy from cell JSON files. Mean model calls: no recovery=15.0, restart=21.0 (+40%), visible retry=15.9, checkpoint=14.9, source rollback=16.1, selective=15.1. Mean wall time: restart=165s, all others 115–123s. Overhead data computed from 46 exercised cells.
- **Source localization accuracy:** 100% across all 46 exercised cells across all three fault types (empty_read: 21/21, wrong_user_id: 16/16, tool_error: 9/9). The detector correctly identifies the injected fault source in every case without any false positives.
- **Literature survey (expanded):** Searched arXiv 2025–2026 for new recovery methods. Found 6 directly relevant papers: "Rollback the World, Keep the Reflection" (2609.18304), "Retry, Switch, or Abstain?" (2608.11977), "Self-Healing Framework for Reliable LLM Agents" (2605.06737), "Outcome Monitors: Recovery Affordances for Silent Tool Failures" (2608.19303), "When Tools Fail: Benchmarking Dynamic Replanning" (2606.05806), "AgentDebugX" (2607.18754).
- **Three new strategies designed and implemented:** rollback with reflection (same state rollback as source rollback, then appends a brief correction note to the restored trajectory so the agent knows why it is re-running — from arXiv 2609.18304), verbal correction (rolls back only one step, then pre-fetches the correct tool result and caches it as verified so the agent receives corrected data in-place without deep rollback — addresses the detection-timing problem found in tool_error trajectory analysis), adaptive (meta-controller routing to empirically best strategy per fault type: empty_read → source rollback, wrong_user_id → restart, tool_error → selective — from arXiv 2608.11977). All three added to tau_recovery.py and confirmed syntax-valid. Phase 4 runs these strategies on seeds 303–305.

---

3. Evidence of Progress

GitHub repository: `plan-my-path/` — contains all source code, data scripts, and documentation. Key files:

- `Backend/research/tau_recovery.py` — RecoveryController adapter (9 strategies, 3 fault types)

- `Backend/research/tau_recovery_matrix.py` — hash-guarded matrix runner
- `Backend/research/build_results_table.py` — results pipeline
- `docs/research/tau-recovery-protocol.json` — frozen experimental protocol
- `docs/research/research-log.md` — detailed session-by-session log
- `docs/research/baselines/results/` — 87 cell JSON files across 9 matrix runs

Results dashboard (live, version 4): https://claude.ai/artifact/QLTJ2fTP5Ugh9x4Yc7ZKNn — interactive reward grid with fault-exercised filter toggle, per-seed breakdowns, and fault coverage table showing exercised/total ratio per seed-fault combination.

Experimental data: 87 cell JSON files, each containing: strategy, fault, seed, task, reward, fault_exercised, exact_source_accuracy, model_calls, tool_executions, validation_calls, cached_reads_reused, wall_time, cost_usd, full event trace, full trajectory.

Summary results table (Phases 1–3, fault_exercised=True, task 0):

Fault	no_recovery	restart	visible_retry	checkpoint	source_rollback	selective
empty_read	0.75 (n=4)	0.67 (n=3)	0.67 (n=3)	0.75 (n=4)	**1.00 (n=3)**	0.50 (n=4)
wrong_user_id	0.67 (n=3)	0.67 (n=3)	**1.00 (n=3)**	0.67 (n=3)	**1.00 (n=3)**	**1.00 (n=3)**
tool_error	1.00 (n=1)	1.00 (n=1)	0.50 (n=2)	0.50 (n=2)	0.50 (n=2)	0.50 (n=2)

N counts reflect seeds with fault_exercised=True; tool_error coverage is low (~33–50%) because the agent sometimes bypasses the injected call.

---

4. Progress Compared with Original Proposal

Original plan (from protocol v1, frozen October 9, 2026):

- 162 controlled cells: 6 strategies × 3 faults × 3 seeds × 3 tasks
- 9 unfaulted reference runs
- Seeds 300–302, tasks 0–2
- Model: gemini/gemini-3.5-flash-lite

Status at midterm:

Planned item	Status
Task 0, seeds 300–302, 6 strategies, 3 faults	✅ Complete (87 cells)
Unfaulted reference runs (task 0)	Partial — seed 301 rate-limited in Phase 1; filled by Phase 4
Tasks 1–2	🔄 Second half (Phase 5)
Seeds 303–305	🔄 Phase 4 partial (57/81 cells; quota limit)
3 new strategies from literature	✅ Designed, implemented, queued in Phase 4
Results pipeline and dashboard	✅ Complete
Source localization analysis	✅ Complete (100% accuracy)
Overhead analysis	✅ Complete
Paper draft	🔄 Second half

Changes to original plan:

1. Seeds expanded from 300–302 to 300–305 (Amendment 3) after observing high variance in tool_error results — needed more data points for reliable means.
2. Model upgraded to gemini-3.6-flash for Phase 4+ after it became available for free-tier. However, gemini-3.6-flash has a stricter daily quota (20 requests/day) than the earlier model, requiring Phase 4 to run across multiple days. Phases 1–3 (gemini-3.5-flash-lite) and 4+ (gemini-3.6-flash) are tracked separately and not pooled.
3. Three new strategies added (not in original protocol) based on literature survey findings.
4. AgentRx static invariant checking was piloted but discontinued — the empty_read fault causes task failure via information deprivation rather than policy violation, making static invariant checking the wrong instrument.

Overall: On schedule. The core experimental infrastructure and Phases 1–3 are complete. Phase 4 is running. The second half proceeds to tasks 1–2, paper writing, and final analysis.

---

5. Challenges and Solutions

Challenge 1: Protocol hash guard abortion (Phase 3) The matrix runner checks the SHA-256 hash of both the adapter file and the protocol JSON before every cell. While Phase 3 was running, the protocol JSON was edited to add seeds 303–305, changing its hash. This triggered `RuntimeError: Adapter or protocol changed during matrix execution`, aborting Phase 3 at 12/36 cells.

Impact: Lost ~24 cells of execution time; Phase 3 results were incomplete for seeds 301–302. *Resolution:* Launched a Phase 3 resume run restricted to seeds 301–302 and the wrong_user_id + tool_error faults (the ones not yet completed). Phase 3 is now complete. Rule documented: never edit `tau_recovery.py` or `tau-recovery-protocol.json` while a matrix run is active.

Challenge 2: Free-tier rate limiting (HTTP 429) During Phase 1 execution, the Gemini free-tier per-minute quota was exceeded mid-run. Several cells were retried but one unfaulted reference run (seed 301) failed to complete.

Impact: Missing unfaulted baseline for seed 301; occasional run delays. *Resolution:* Added a 6-second fixed pacing delay before every agent/user generation call (Amendment 1 to protocol). The delay is identical across all strategies so it does not confound comparisons. The missing seed 301 unfaulted run will be filled by Phase 4.

Challenge 3: AgentRx integration — wrong instrument Attempted to use the AgentRx static invariant checking pipeline to identify agent policy violations in the trajectories. After debugging the copilot CLI (hardcoded model name issue), successfully ran it across all available trajectories and received 0 violations.

Impact: Several hours of integration work that did not yield findings. *Resolution:* Determined that empty_read causes task failure through information deprivation — the agent proceeds on empty user data without violating any stated policy rules — so static invariant checking cannot detect this class of failure. AgentRx discontinued for this evaluation; method noted as a limitation. The debugging work produced a patched copilot_cli.py that is operational for other use.

Challenge 4: Fault coverage variability for tool_error The tool_error fault fires only when the agent calls `get_user_details` directly. In ~50–67% of runs, the agent uses `find_user_id_by_name_zip` instead, bypassing the injection point entirely. This means `fault_exercised=False` for those cells, and they cannot be used in the primary analysis.

Impact: Low effective N for tool_error (1–2 exercised cells per seed vs 6 planned), making tool_error means unreliable. *Resolution:* Tracking `fault_exercised` per cell and conditioning all primary analysis on it. Increasing seeds from 3 to 6 (Phase 4) to raise the expected exercised count. Also noted as a study limitation: fault coverage is task-instance-dependent, not strategy-dependent.

Challenge 5: Result misinterpretation (tool_error) Initial reading of the tool_error grid suggested no_recovery = 1.0 was the best strategy, making recovery seem counterproductive.

Impact: Would have led to a wrong conclusion in the paper. *Resolution:* Confirmed that no_recovery's high reward came from runs where `fault_exercised=False` — the fault never fired. Corrected the analysis to condition strictly on fault_exercised=True. The corrected finding is that tool_error results are highly seed-dependent and detection-timing-driven.

Challenge 6: Fault coverage does not imply fault impact Two no_recovery cells (seed=301, wrong_user_id and tool_error) achieved reward=1.0 with fault_exercised=True, suggesting the agent recovered without any infrastructure.

Impact: If genuine, this would mean no_recovery outperforms active strategies on certain seeds, undermining the comparison. *Resolution:* Full trajectory read of both cells identified the mechanism. Wrong_user_id: the agent called find_user_id_by_name_zip at step 4 and obtained the correct user ID before get_user_details fired the fault at step 8; the corrupted sentinel appeared in the trajectory but was never passed to any downstream action — the agent used the earlier value. Tool_error: the agent retried the failed call at the next step; since the fault fires only once per run, the retry returned authentic data. Neither case is self-correction. Consequence: fault_exercised=True is necessary but not sufficient to confirm fault impact. A stricter metric, fault_impacted, would require the corrupted value to appear in a downstream action. We report this as a study limitation.

---

6. Current Project Status

At the midpoint of the semester, the project has completed its core infrastructure and all Phase 1–3 experiments. Specific accomplishments:

- **RecoveryController** with 9 strategies (6 original + 3 new), 3 fault types, DAG dependency tracking, and verified cache is fully implemented and tested.

- **87 experimental cells** collected across task 0, seeds 300–302, all 6 strategies, all 3 faults. Each cell contains full trajectory, event trace, and rich metrics.
- **Source localization:** 100% accurate — the detector correctly identifies the injected fault source in every exercised run.
- **Overhead:** Restart adds ~40% model calls and wall time. Other strategies add ⩽2 calls (~13% overhead), making them practical.
- **Key finding per fault type:**
  - *empty_read:* source rollback dominates (mean 1.00, n=3 seeds). Selective underperforms (mean 0.50, n=4) because it invalidates too aggressively under conservative dependency rules. Checkpoint is brittle (0.0 on seed 301; depends on fault timing relative to 4-step boundaries).
  - *wrong_user_id:* Visible retry, source rollback, and selective all achieve 1.00 (n=3). Restart and checkpoint show seed-dependent brittleness (0.0 on seed 302). Notably, no recovery achieved 1.0 on seed 301 with the fault exercised — trajectory analysis (October 12) found both are structural bypasses, not genuine self-correction: for wrong_user_id, the agent had already obtained the correct user ID via find_user_id_by_name_zip before the fault fired and the corrupted sentinel was never used downstream; for tool_error, the agent's natural retry succeeded because the protocol injects the fault only once per run. Key implication: fault_exercised=True confirms the fault fired but does not confirm it changed the agent's behavior (see Challenge 6).
  - *tool_error:* Results are highly variable: selective=1.0 on seed 300 (detected at step 14 with few steps remaining), selective=0.0 on seed 301 (detected earlier, ran out of conversation turns). This detection-timing effect explains all tool_error variance — it is not a mechanism advantage. No dominant strategy identified; Phase 4 (seeds 303–305, larger N) needed for reliable conclusions.
- **Phase 4 partially ran (57 cells completed before hitting the Gemini free-tier daily quota limit of 20 requests/day; resumes when quota resets). The 3 new strategies are queued for the next Phase 4 batch.** Seeds 303–305 are queued; the gemini-3.6-flash model has a free-tier limit of 20 requests/day which was exhausted during the first Phase 4 run. Remaining cells will run after the daily quota resets.

Remaining work: completing Phase 4 (resuming after quota reset), tasks 1–2 (Phase 5), and paper writing — all second-half items.

---

7. Plan for the Second Half of the Semester

Weeks 8–9: Complete Phase 4 and Fix Failed Cells

- Resume Phase 4 after daily quota resets (gemini-3.6-flash free tier: 20 requests/day). Analyze seeds 303–305 results and first runs of the 3 new strategies.
- Re-run 4 failed cells (null reward): restart/seed=302/empty_read, no_recovery/seed=301/empty_read, selective/seed=302/empty_read, visible_retry/seed=301/tool_error.
- ✅ Trajectory analysis complete (October 12): no_recovery + wrong_user_id succeeds via structural bypass (agent obtained user ID from find_user_id_by_name_zip before fault fired; corrupted value never used downstream); no_recovery + tool_error succeeds via natural retry (fault fires once per run; retry returns authentic data). Neither is genuine self-correction. New study limitation added: fault_exercised ≠ fault_impacted. See Challenge 6.
- Compare the 3 new strategies (rollback_with_reflection, verbal_correction, adaptive) against the original 6 per fault type.

Weeks 10–11: Phase 5 — Tasks 1 and 2

- Run Phase 5 matrix: tasks 1 and 2, seeds 300–305, all 9 strategies, all 3 faults.
- Analysis focuses on whether findings from task 0 generalize. Source_rollback's dominance on empty_read and the tool_error variance pattern are the key claims to test.

Weeks 12–13: Paper Writing

- Write methodology section: experimental design, fault injection, RecoveryController, protocol, metrics.
- Write results section: per-fault reward tables, overhead table, source localization, trajectory analysis.
- Write discussion: detection-timing insight for tool_error, verbal_correction hypothesis test, adaptive as oracle bound, checkpoint brittleness explanation, limitations (fault coverage, single benchmark, single model family).

Weeks 14–15: Finalization

- Final results table with all phases combined.
- Figures: reward heatmap per fault × strategy, overhead scatter, detection-step vs. reward scatter for tool_error.
- Proof-read and submit paper.

- Prepare final presentation.